

# Individual Research Gap Analysis

## 1. Towards Secure Cloud-Native Computing: Unveiling Kubernetes Misconfigurations with Large Language Models

### Research Gaps:

This study builds a structured taxonomy of Kubernetes misconfigurations and shows that general-purpose LLMs detect a broader range of issues than static rule-based scanners such as Checkov and KubeLinter. However, the work is detection-only, offers no automated repair, and is confined entirely to Kubernetes, with no coverage of Terraform, CloudFormation, CDK, or AWS SAM. It also relies on ungrounded commercial LLMs without retrieval-based knowledge injection and provides no mechanism for correlating detected misconfigurations with live cloud telemetry, leaving a clear gap for the coordinated, multi-format, telemetry-aware validation and security-assessment agents this project proposes.

## 2. LLMSecConfig: An LLM-Based Approach for Fixing Software Container Misconfigurations

### Research Gaps:

LLMSecConfig pairs static analysis tools with LLMs through Retrieval-Augmented Generation in an iterative detect-retrieve-patch-revalidate loop, achieving a 94% repair success rate on real Kubernetes configurations, far surpassing a plain-LLM baseline of roughly 40%. Despite this strength, the framework acts reactively, after a misconfiguration is already flagged, rather than validating infrastructure proactively before deployment. Its scope is also limited to container and Kubernetes configurations, excluding Terraform, CloudFormation, CDK, and serverless formats central to this project, and repair quality depends entirely on the underlying scanner's coverage, leaving no path toward pre-deployment, multi-format, telemetry-integrated validation.

## 3. ARPaCCino: An Agentic-RAG for Policy as Code Compliance

### Research Gaps:

ARPaCCino shows that an agentic-RAG architecture combining LLMs, retrieval, and tool-based validation substantially outperforms plain LLM and plain RAG approaches at converting natural-language policy into enforceable Rego rules and checking IaC compliance. However, it depends on manually curated, framework-specific knowledge bases that do not scale automatically to new IaC technologies, and its iterative agentic loops raise latency and cost. Crucially, ARPaCCino stops at policy compliance and does not extend to deployment verification, configuration-drift correlation with live telemetry, or cost analysis, capabilities this project's multi-agent framework must integrate into a single coordinated workflow.

## **4. Kubernetes Misconfigurations in the Wild: Taxonomy, Evolution, and Automated Repair with Large Language Models**

### **Research Gaps:**

This paper builds a longitudinal, multi-LLM-derived taxonomy of Kubernetes misconfigurations and proposes a hybrid LLM plus deterministic schema-enforcement repair method achieving roughly 98.5% correction accuracy. While valuable for understanding how misconfiguration patterns evolve, the study remains Kubernetes-only, its taxonomy construction risks categorization inconsistency across models, and it includes no retrieval-augmented grounding or connection to live cloud state. Being retrospective and single-platform, it cannot inform real-time, multi-format validation or correlate static misconfiguration patterns with runtime telemetry across the broader AWS service set that this project's framework is designed to cover.

## **5. LM-Based Misconfiguration Detection for AWS Serverless Computing**

### **Research Gaps:**

SlsDetector introduces a zero-shot prompting framework using multi-dimensional constraints and chain-of-thought reasoning to detect AWS Serverless Application Model misconfigurations, achieving strong F1 performance across multiple LLM backends. However, it performs detection only, with no automated repair, no retrieval grounding, and evaluation limited to a small dataset of 110 files from a single cloud provider's serverless schema. This narrow, single-format scope cannot generalize to the multi-format Terraform, CloudFormation, CDK, Kubernetes, and SAM validation and repair pipeline this project targets, nor does it integrate runtime telemetry or coordinate with agents handling other IaC formats in one workflow.

## **Combined Research Gap Analysis**

Across these five studies, LLM-assisted Kubernetes and serverless misconfiguration research shows strong progress on isolated capabilities: taxonomy-driven detection, RAG-enhanced repair, agentic policy compliance, and zero-shot serverless detection. Yet each is confined to a single platform, Kubernetes or AWS SAM, and a single stage of the validation lifecycle, detection, repair, or compliance alone. None combines multi-format coverage across Terraform, CloudFormation, CDK, Kubernetes, and SAM; none correlates static misconfiguration detection with live runtime telemetry such as AWS CloudWatch metrics; and none coordinates detection, repair, compliance, drift detection, and cost analysis through a single multi-agent architecture. This composite gap, fragmented platform scope, a static-to-runtime disconnect, and uncoordinated single-task tools, directly motivates the proposed Telemetry-Aware Multi-Agent Infrastructure-as-Code Assurance System, which unifies parsing, security assessment, deployment verification, drift detection, and cost analysis into one explainable, telemetry-grounded workflow for smart-manufacturing cloud environments.